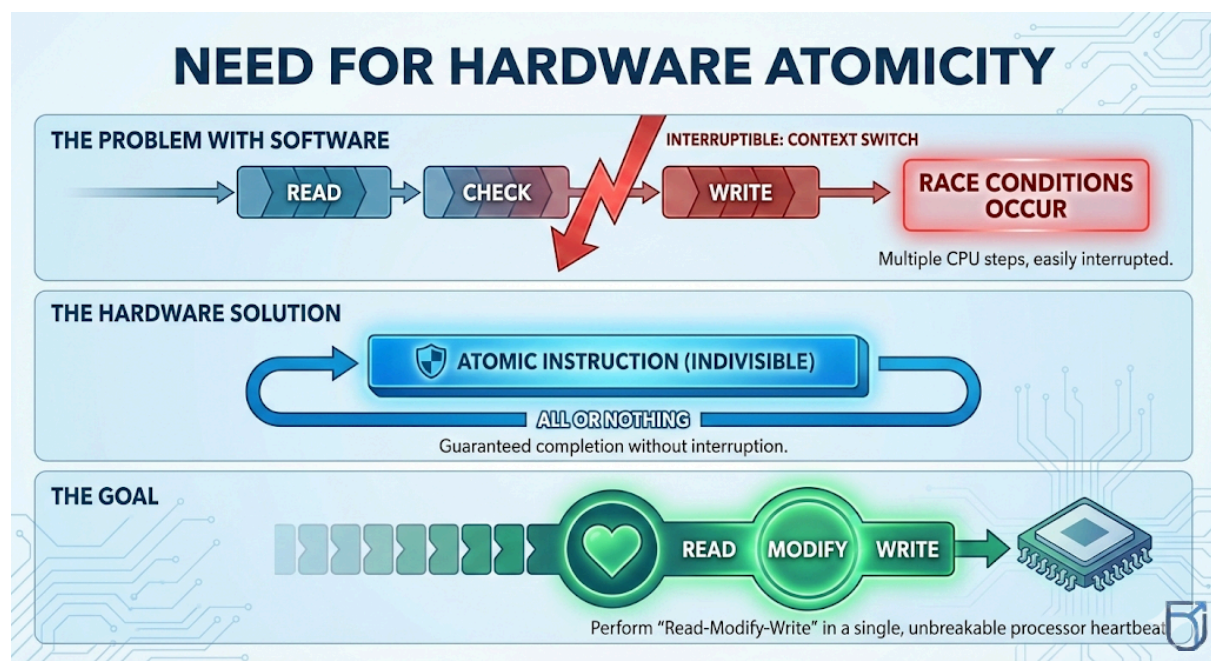


Lecture 12 NOTES – Process Synchronization III: Hardware Support and Mutex Locks

Need for Hardware Atomicity

The Problem with Software:

- As we saw in the "Bank Balance" example, high-level code is broken down into multiple CPU steps: **Read** → **Check** → **Write**.
- These steps are **interruptible**. A context switch can happen in the middle (e.g., after reading but before writing), causing race conditions.



The Hardware Solution:

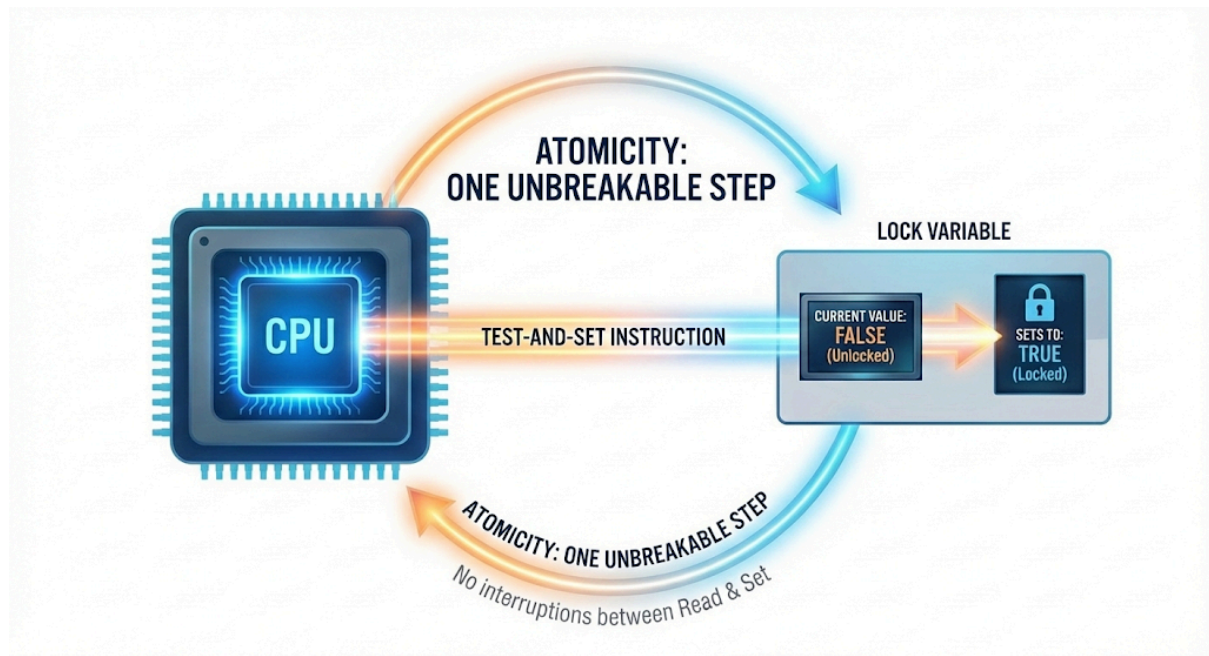
- The CPU provides special instructions that are **Atomic** (Indivisible).
- **Meaning:** When this instruction starts, the hardware guarantees it will finish **completely** without any interruptions or context switches. It's "all or nothing."

Goal: To perform the "Read-Modify-Write" cycle in a single, unbreakable heartbeat of the processor.

Hardware Synchronization Primitives

The most common hardware tool is the **Test-and-Set (TAS)** instruction.

1. Test-and-Set (TAS)



The Concept

- **What it is:** A special, ultra-fast instruction built directly into the CPU.
- **What it does:** It looks at a lock variable, reads its current value, and *simultaneously* sets it to **TRUE** (Locked).
- **The "Magic" (Atomicity):** The most important part is that this happens as **one single, unbreakable step**. No other process can interrupt or cut in between the reading and the setting. It is "all or nothing."

The Operation (How it works internally) Imagine a single hardware command that performs these three steps instantly:

1. **Read:** It checks the current status of the lock (Open or Closed).
2. **Set:** It immediately changes the status to **Closed (True)**, no matter what it was before.
3. **Return:** It tells you what the status *was* before you touched it.

2. Using TAS to Build a "Spinlock"

We use this hardware instruction to build a simple locking mechanism called a **Spinlock**. Here is how the logic works without code:

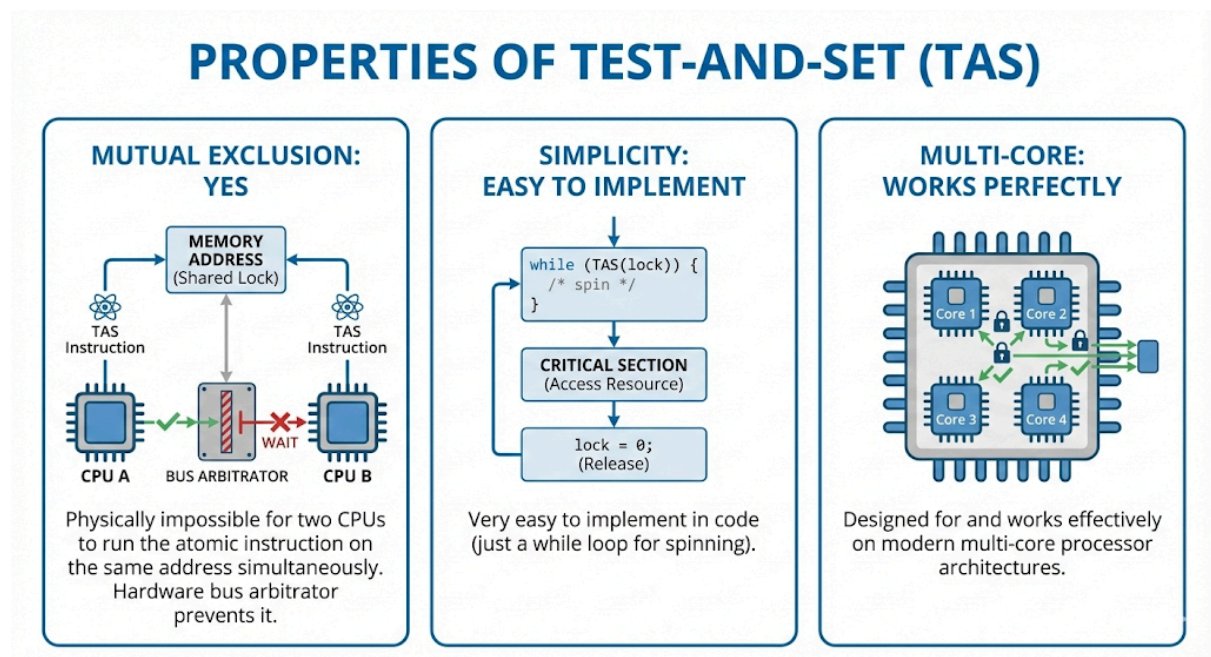
A. Acquiring the Lock (Entry Section) To get into the Critical Section, a process runs a loop that keeps asking the CPU to run **Test-and-Set** on the lock variable.

- **Scenario 1: The Lock is FREE (False)**
 - The instruction reads "False" (Free).
 - It instantly sets it to "True" (Locked).
 - It returns "False".
 - **Result:** You see that it was free, so you know you successfully locked it. You **Enter**.
- **Scenario 2: The Lock is BUSY (True)**
 - The instruction reads "True" (Busy).
 - It sets it to "True" (keeps it Locked).
 - It returns "True".
 - **Result:** You see that it was already busy. You stay in the loop and try again (**Spin**).

B. Releasing the Lock (Exit Section)

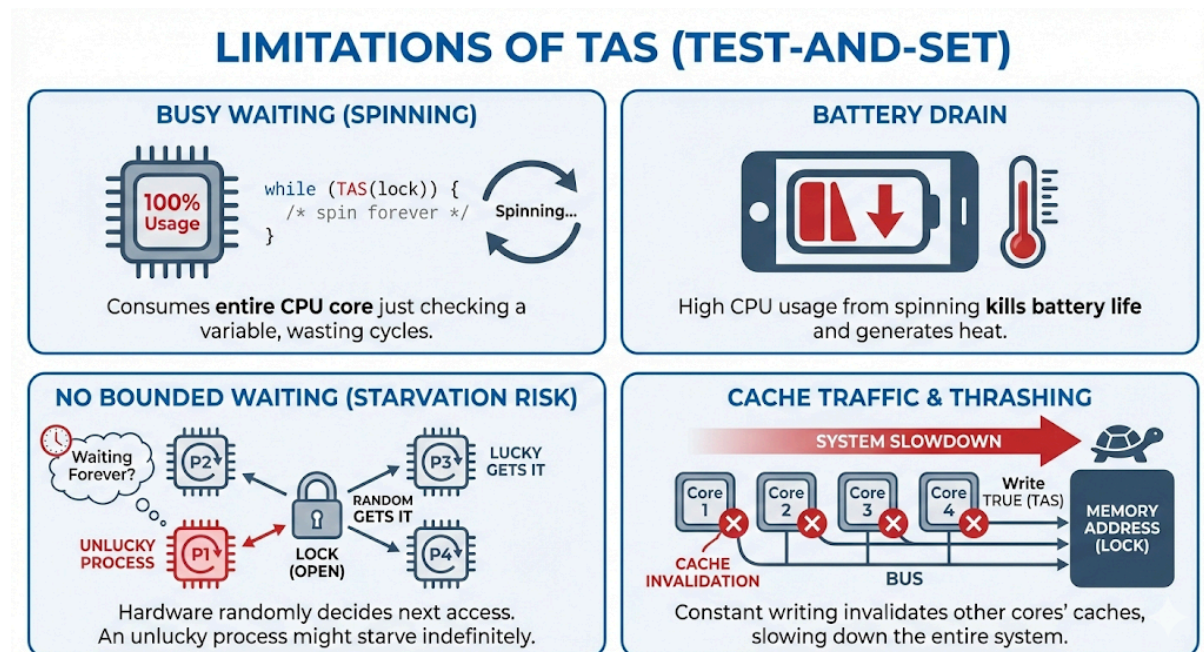
- This part is very simple. You just set the lock variable back to **FALSE** (Free).
- This instantly opens the door for the next process that is currently spinning.

Properties of TAS



- **Mutual Exclusion: Yes.** It is physically impossible for two CPUs to run the atomic instruction on the same address at the exact same time. The hardware bus arbitrator prevents it.
- **Simplicity:** It is very easy to implement in code (just a **while** loop).
- **Multi-Core:** Works perfectly on multi-core processors.

Limitations of TAS



- **Busy Waiting (Spinning):** The waiting process loops forever (`while...`), consuming 100% of the CPU core just to check a variable.
- **Battery Drain:** High CPU usage kills battery life.
- **No Bounded Waiting:**
 - **Starvation Risk:** If 5 processes are spinning, and the lock opens, the hardware randomly decides who gets it next. An unlucky process might wait forever.
- **Cache Traffic:** Constantly writing `TRUE` to the memory address invalidates the caches of all other cores, slowing down the entire system (known as "cache thrashing").

Code Implementation:

```
#include <stdio.h>  
#include <pthread.h>  
#include <stdatomic.h>  
#include <unistd.h>  
  
// Shared Spinlock (atomic_flag is the only type guaranteed to be
```



```

lock-free)
atomic_flag lock = ATOMIC_FLAG_INIT;
// Shared Resource
int shared_counter = 0;

// Spinlock Acquire Function (The "Test-And-Set" loop)
void acquire_spinlock() {
    // atomic_flag_test_and_set returns the OLD value.
    // If it returns true, it means someone else holds it → We spin.
    // If it returns false, we just set it to true → We enter.
    while (atomic_flag_test_and_set(&lock)) {
        // Busy wait (Spinning)
    }
}

// Spinlock Release Function
void release_spinlock() {
    atomic_flag_clear(&lock);
}

// Thread Function
void* thread_function(void* arg) {
    int id = *(int*)arg;

    for (int i = 0; i < 100000; i++) {
        acquire_spinlock();

        // --- CRITICAL SECTION ---
        shared_counter++;
        // -----

        release_spinlock();
    }

    return NULL;
}

```

```
int main() {
    pthread_t t1, t2;
    int id1 = 1, id2 = 2;

    printf("Starting Spinlock Test ... \n");

    // Create two threads
    pthread_create(&t1, NULL, thread_function, &id1);
    pthread_create(&t2, NULL, thread_function, &id2);

    // Wait for them to finish
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

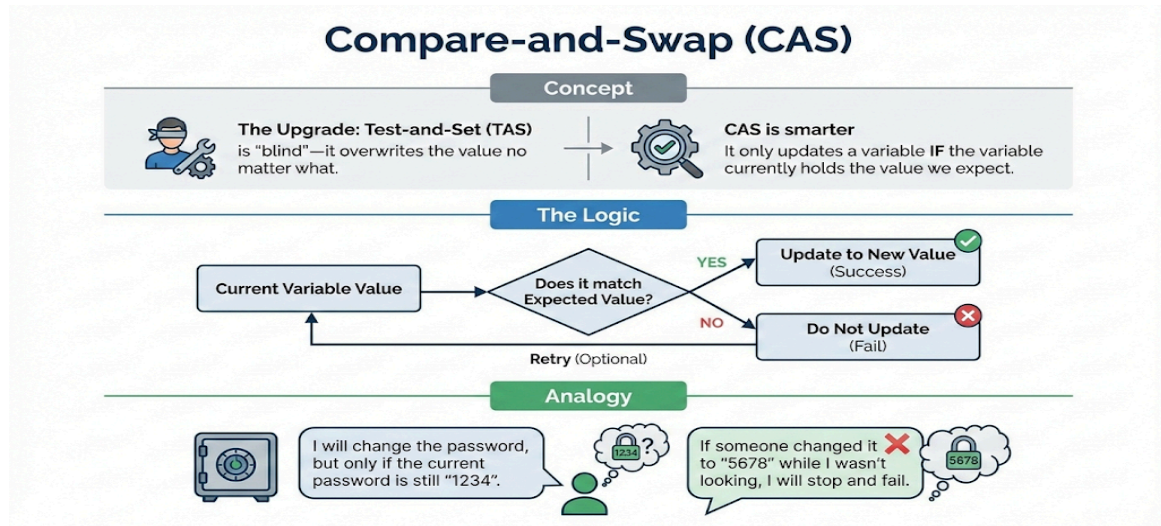
    // If synchronization works, counter should be exactly 200,000
    printf("Final Counter Value: %d (Expected: 200000)\n",
shared_counter);

    return 0;
}
```

Output:

```
Starting Spinlock Test ...
Final Counter Value: 200000 (Expected: 200000)
```

Compare-and-Swap (CAS)



Concept

- **The Upgrade:** Test-and-Set (TAS) is "blind"—it overwrites the value no matter what. **CAS** is smarter.
- **The Logic:** It only updates a variable **IF** the variable currently holds the value we expect.
- **Analogy:** "I will change the password, but *only if* the current password is still '1234'. If someone changed it to '5678' while I wasn't looking, I will stop and fail."

CAS Operation

This is an atomic hardware instruction that takes three inputs:

1. **Address:** Where the data is.
 2. **Expected Value:** What we think is there.
 3. **New Value:** What we want to write.
- **Rule:** If **Current Value == Expected Value**, then **Current Value = New Value**.
 - **Result:** It returns true (success) or false (failure), allowing the software to decide what to do next (usually retry).

Code Implementation:

```
#include <stdio.h>
#include <pthread.h>
#include <stdatomic.h>

// Shared Atomic Variable
_Atomic int shared_counter = 0;

// Atomic Increment using Compare-And-Swap (CAS)
void atomic_increment() {
    int old_val, new_val;

    // Optimistic Loop
    do {
        old_val = atomic_load(&shared_counter); // Step 1: Read
        new_val = old_val + 1;                  // Step 2: Calculate
new value locally

        // Step 3: CAS
        // Checks if shared_counter is still 'old_val'.
        // If YES → Updates to 'new_val' and returns true (Loop
ends).
        // If NO → Updates 'old_val' with current value and returns
false (Loop retries).
    } while (!atomic_compare_exchange_strong(&shared_counter,
&old_val, new_val));
}

void* thread_function(void* arg) {
    for (int i = 0; i < 100000; i++) {
        atomic_increment();
    }
}
```

```
    }  
    return NULL;  
}  
  
int main() {  
    pthread_t t1, t2;  
  
    printf("Starting CAS Optimistic Locking Test ... \n");  
  
    pthread_create(&t1, NULL, thread_function, NULL);  
    pthread_create(&t2, NULL, thread_function, NULL);  
  
    pthread_join(t1, NULL);  
    pthread_join(t2, NULL);  
  
    printf("Final Counter Value: %d (Expected: 200000)\n",  
atomic_load(&shared_counter));  
  
    return 0;  
}
```

Output:

```
Starting CAS Optimistic Locking Test ...  
Final Counter Value: 200000 (Expected: 200000)
```

CAS vs. TAS

Feature	Test-and-Set (TAS)	Compare-and-Swap (CAS)
Philosophy	Pessimistic / Aggressive. "I am taking this lock now."	Optimistic / Careful. "I will try to update this, assuming no one else has."
Operation	Unconditional Write. Sets variable to 1 no matter what.	Conditional Write. Sets variable only if it equals the expected value.
Complexity	Simple. Used mostly for basic Spinlocks.	More complex. Used for "Lock-Free" algorithms and atomic counters.
Power	Lower. Can only toggle a bit.	Higher. Can check for specific values (like version numbers).

Optimistic Concurrency (The "Lock-Free" Dream)

- **Traditional Locking:** "I will lock the door so no one enters." (Pessimistic).
- **Optimistic (CAS):** "I won't lock the door. I will just try to do my work quickly. If I finish and realize someone else messed with my data, I will just redo my work."
- **Benefit:** Extremely fast because you avoid the overhead of heavy locks.

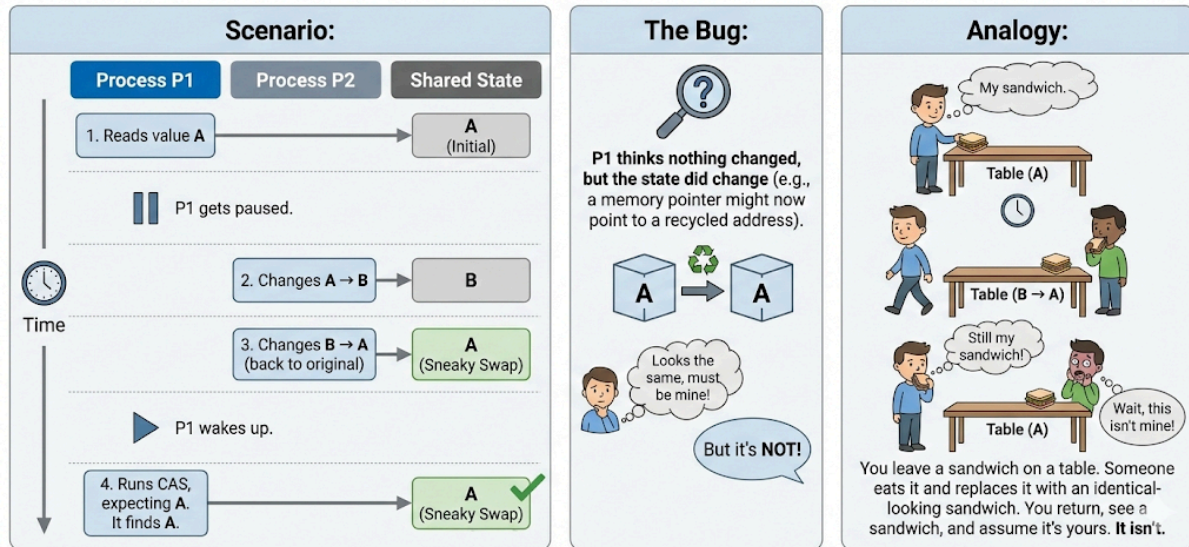
CAS Use Case

- **Atomic Counters:** Increasing a viewer count on a website without locking the whole database.
- **Concurrent Data Structures:** Building Queues or Stacks that multiple threads can use without ever stopping each other.

The ABA Problem (The "Sneaky Swap")

The ABA Problem (The "Sneaky Swap")

Threads can use without ever stopping each other. This is a famous bug specific to CAS.



This is a famous bug specific to CAS.

- **Scenario:**
 1. Process P1 reads value **A**.
 2. P1 gets paused.
 3. Process P2 changes **A** → **B**.
 4. Process P2 changes **B** → **A** (back to original).
 5. P1 wakes up. It runs CAS, expecting **A**. It finds **A**.
 6. **The Bug:** P1 thinks *nothing* changed, but the state *did* change (e.g., a memory pointer might now point to a recycled address).
- **Analogy:** You leave a sandwich on a table. Someone eats it and replaces it with an identical-looking sandwich. You return, see a sandwich, and assume it's yours. It isn't.
- **Solution:** Use **Version Numbers** (e.g., A1 → B2 → A3). Even if the value is A, the version (3) tells you it changed.

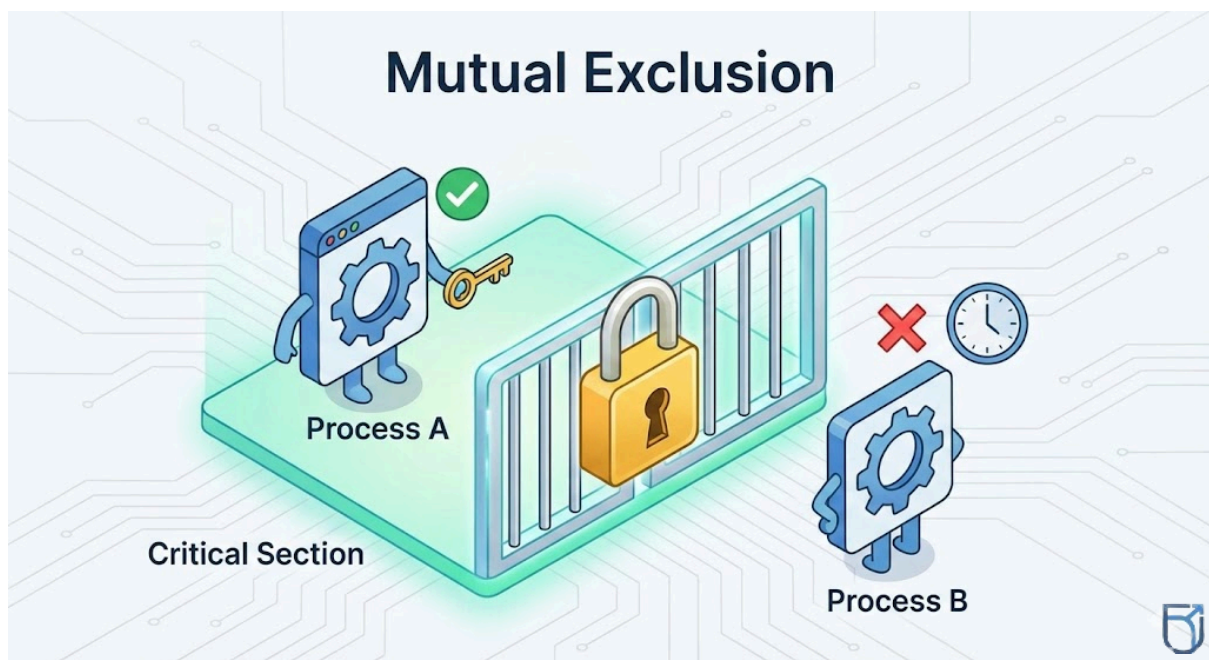
Efficiency Problem with Spinlocks

- **The Issue:** Both TAS and CAS are used to make **Spinlocks**.
- **Why it sucks:** A waiting thread runs in a **while(true)** loop.
 - It burns 100% CPU cycles doing nothing.
 - It heats up the processor and drains the battery.
- **The Need:** We need a way to **Blocking (Sleeping)** instead of spinning. We want the thread to go to sleep and only wake up when the lock is free.

Mutex Locks

Introduction to Mutex

- **Definition:** Short for **Mutual Exclusion**.
- **What it is:** A high-level software tool provided by the OS (unlike TAS/CAS which are hardware instructions).
- **The Upgrade:** It uses hardware instructions (TAS/CAS) internally to set the lock, but adds a **Queue** and **Sleep** mechanism to save CPU power.

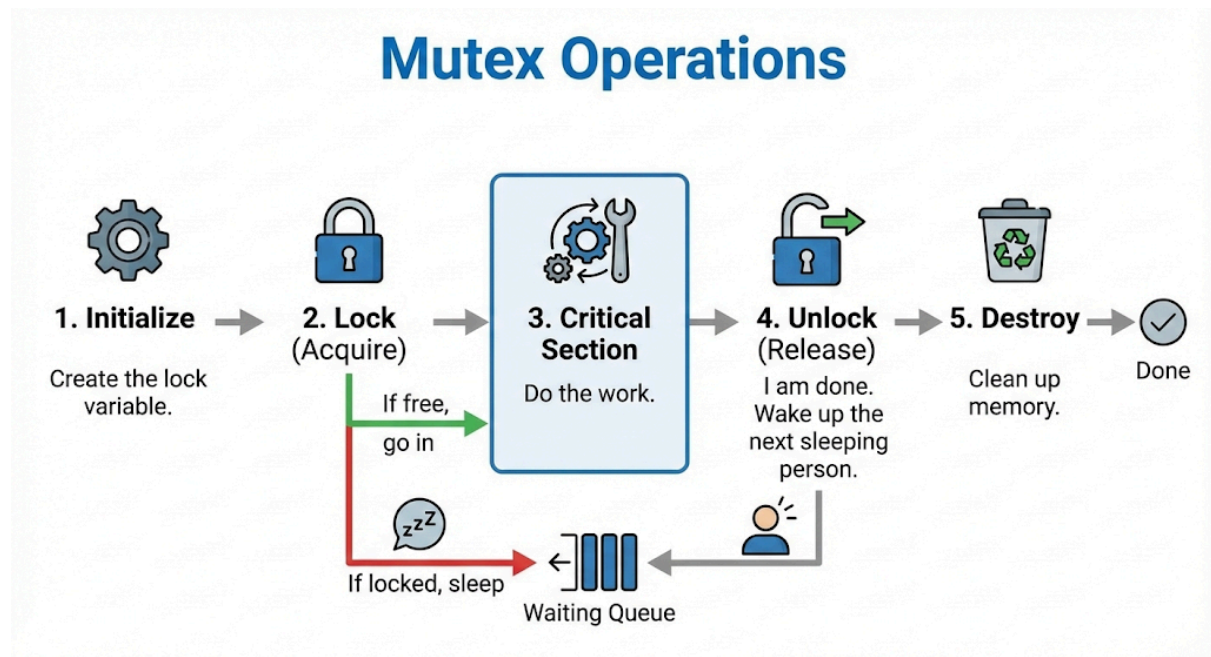


Spinlock vs. Mutex (Core Idea)

Feature	Spinlock	Mutex
Behavior	"Are we there yet? Are we there yet?" (Constant checking).	"Wake me up when we get there." (Sleeps).
CPU Usage	High (Wastes cycles).	Low (Thread is removed from CPU).

Context Switch	None (Thread stays on CPU).	Yes (OS switches to another thread).
Use Case	Very short waits (Nanoseconds).	Long waits (File I/O, Database).

Mutex Operations

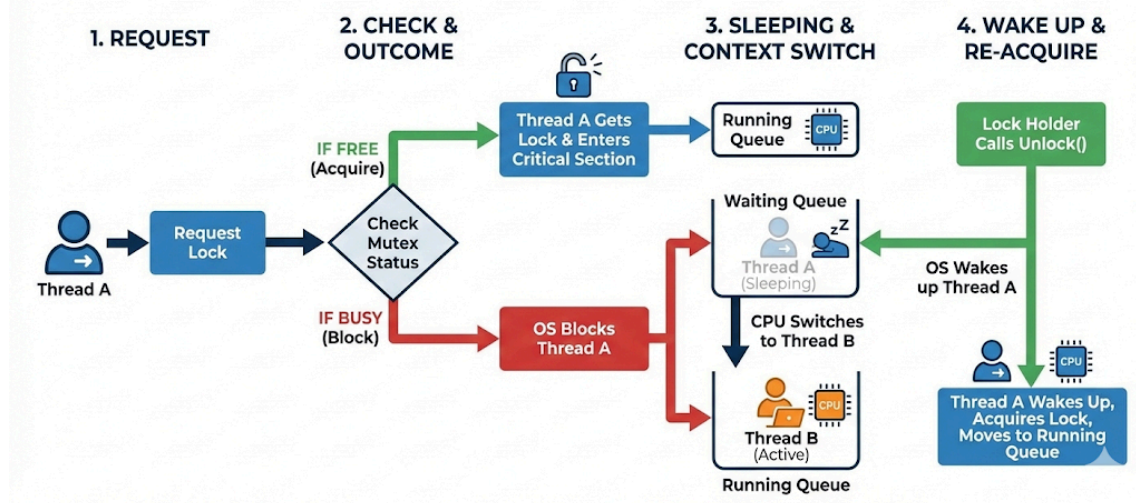


1. **Initialize**: Create the lock variable.
2. **Lock (Acquire)**: "I want to enter." If free, go in. If locked, sleep.
3. **Critical Section**: Do the work.
4. **Unlock (Release)**: "I am done." Wake up the next sleeping person.
5. **Destroy**: Clean up memory.

How Mutex Works (The Flow)

1. **Request**: Thread A asks for the lock.
2. **Check**:
 - **If Free**: Thread A gets the lock and enters.
 - **If Busy**: The OS **blocks** Thread A.

How Mutex Works (The Flow)



3. **Sleeping:** Thread A is moved from the "Running Queue" to a **Waiting Queue**. The CPU starts working on Thread B (doing useful work).
4. **Wake Up:** When the lock holder calls `Unlock()`, the OS looks at the Waiting Queue, picks Thread A, wakes it up, and gives it the lock.

Mutex Internal Implementation

Modern Mutexes use a **Hybrid Design** (often called "Futex" in Linux):

- **Fast Path (User Space):**
 - The thread tries to grab the lock using an atomic instruction (CAS).
 - If successful, it enters immediately. **Zero OS overhead**. Super fast.
- **Slow Path (Kernel Space):**
 - If the lock is busy, *only then* does it make a "System Call" to the OS kernel to be put to sleep.
 - This balances speed (fast path) with efficiency (slow path).
- **Fairness:** The kernel usually maintains the Waiting Queue as **FIFO** (First-In, First-Out) so the longest-waiting thread gets the lock next.

Code Implementation:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
```

```

// The Mutex Lock
pthread_mutex_t lock;

// Shared Resource
int shared_counter = 0;

void* thread_function(void* arg) {
    int id = *(int*)arg;

    for (int i = 0; i < 100000; i++) {
        // 1. Acquire Lock (Sleeps if busy)
        pthread_mutex_lock(&lock);

        // --- CRITICAL SECTION ---
        shared_counter++;
        // -----

        // 2. Release Lock (Wakes up waiting threads)
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;
    int id1 = 1, id2 = 2;

    // Initialize the mutex
    if (pthread_mutex_init(&lock, NULL) != 0) {
        printf("Mutex init has failed\n");
        return 1;
    }
}

```

```
printf("Starting Mutex Test ... \n");

pthread_create(&t1, NULL, thread_function, &id1);
pthread_create(&t2, NULL, thread_function, &id2);

pthread_join(t1, NULL);
pthread_join(t2, NULL);

printf("Final Counter Value: %d (Expected: 200000)\n",
shared_counter);

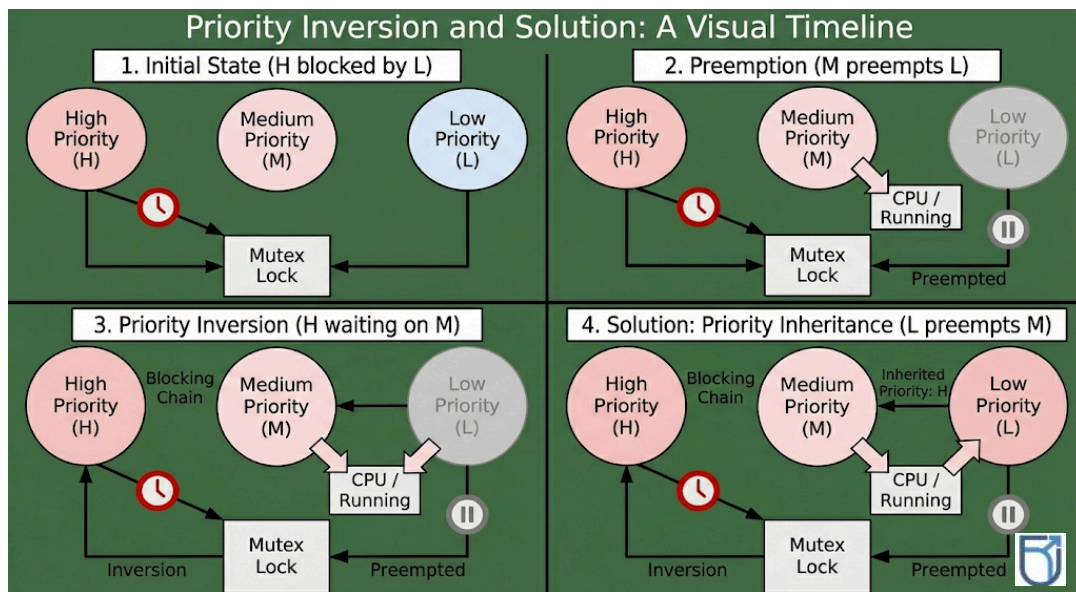
// Destroy the mutex to free resources
pthread_mutex_destroy(&lock);

return 0;
}
```

Output:

```
Starting Mutex Test ...
Final Counter Value: 200000 (Expected: 200000)
```

Priority Inversion



The Problem: This is a famous bug where the scheduling rules break down because of locks. It usually involves three threads: **High (H)**, **Medium (M)**, and **Low (L)** priority.

1. **L** runs and grabs a lock (Mutex).
2. **H** wakes up and tries to grab the same lock. It waits (blocks) because **L** has it.
3. **M** wakes up. Since **M** has higher priority than **L** (and **M** doesn't need the lock), the scheduler lets **M** preempt **L**.
4. **The Result:** **L** cannot run (because of **M**), so it cannot release the lock. **H** is stuck waiting for **L**, which is stuck waiting for **M**. Effectively, the **Medium** thread is blocking the **High** thread!

The Solution: Priority Inheritance

- **How it works:** If a High-priority thread is waiting for a Low-priority thread, the OS temporarily boosts the Low thread's priority to "High."
- **Analogy:** If a VIP is stuck behind a slow car in a one-lane tunnel, the police give the slow car a siren and escort so it can get out of the way faster.

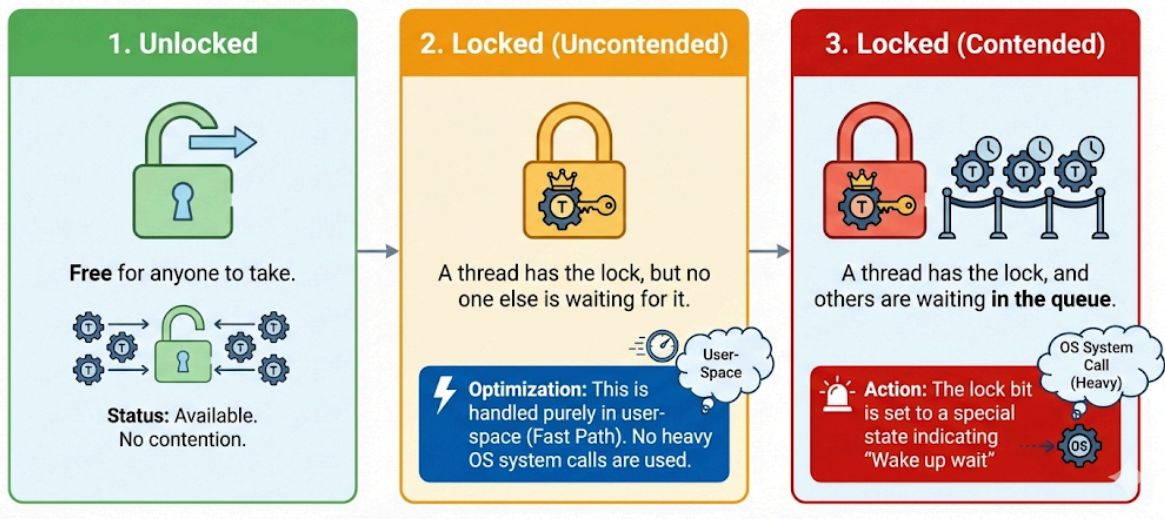
Mutex States

A Mutex isn't just "Open" or "Closed." Modern OS mutexes have three internal states to optimize performance:

1. **Unlocked:** Free for anyone to take.
2. **Locked (Uncontended):** A thread has the lock, but *no one else* is waiting for it.

- **Optimization:** This is handled purely in user-space (Fast Path). No heavy OS system calls are used.

Modern OS Mutex States: Beyond Open and Closed

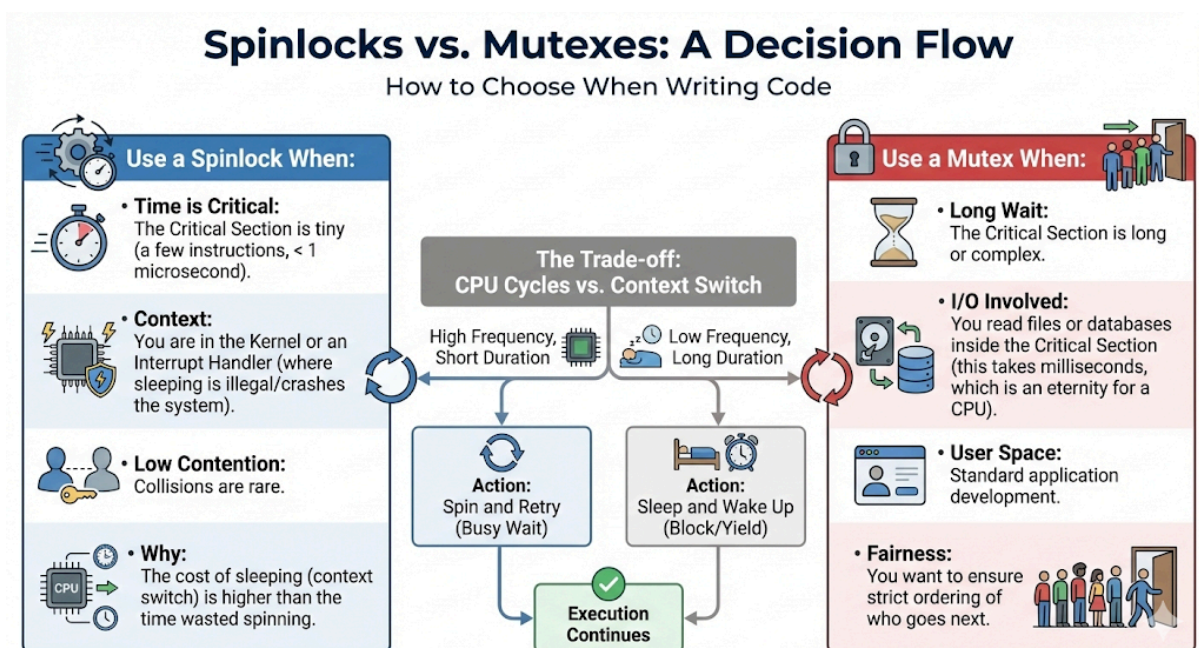


3. **Locked (Contended):** A thread has the lock, and *others* are waiting in the queue.

- **Action:** The lock bit is set to a special state indicating "Wake up waiters when done." This forces the slow path (System Call) on unlock.

Spinlocks vs. Mutexes (Decision Flow)

When writing code, how do you choose?



Use a Spinlock When:

- **Time is Critical:** The Critical Section is tiny (a few instructions, < 1 microsecond).
- **Context:** You are in the **Kernel** or an **Interrupt Handler** (where sleeping is illegal/crashes the system).
- **Low Contention:** Collisions are rare.
- *Why:* The cost of sleeping (context switch) is higher than the time wasted spinning.

Use a Mutex When:

- **Long Wait:** The Critical Section is long or complex.
- **I/O Involved:** You read files or databases inside the Critical Section (this takes milliseconds, which is an eternity for a CPU).
- **User Space:** Standard application development.
- **Fairness:** You want to ensure strict ordering of who goes next.

Hybrid / Adaptive Locks

Modern systems (like Linux and macOS) don't force you to choose one or the other strictly. They use **Adaptive Mutexes**.

How it works:

1. **Phase 1 (Spin):** When a thread tries to lock, it spins for a short time (e.g., a few microseconds) hoping the owner finishes quickly.
2. **Phase 2 (Block):** If the lock is still busy after the spin timeout, the thread gives up spinning and asks the OS to put it to sleep.

Examples:

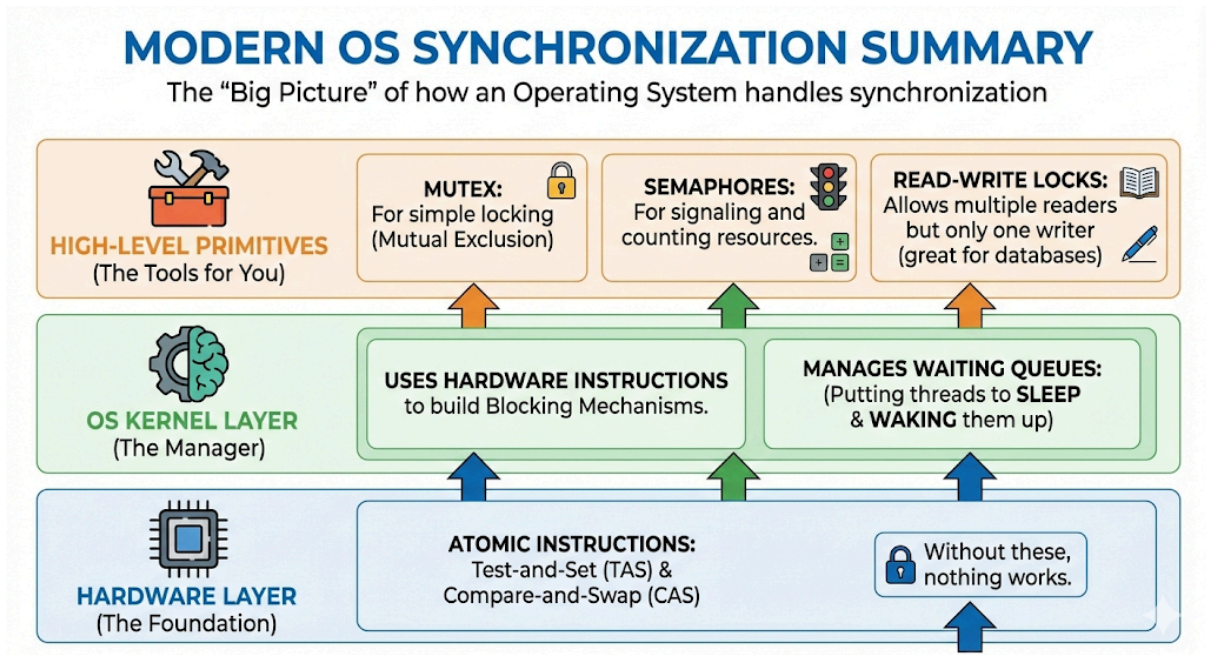
- **Linux Adaptive Mutex:** Spins only if the lock owner is currently running on another CPU core.
- **macOS `os_unfair_lock`:** An extremely fast lock that replaces older spinlocks.

Modern OS Synchronization Summary

This is the "Big Picture" of how an Operating System handles synchronization:

1. **Hardware Layer (The Foundation):**
 - Provides **Atomic Instructions** like Test-and-Set (TAS) and Compare-and-Swap (CAS). Without these, nothing works.
2. **OS Kernel Layer (The Manager):**

- Uses hardware instructions to build **Blocking Mechanisms**.
- Manages **Waiting Queues** (putting threads to sleep and waking them up).



3. High-Level Primitives (The Tools for You):

- **Mutex:** For simple locking (Mutual Exclusion).
- **Semaphores:** For signaling and counting resources.
- **Read-Write Locks:** Allows multiple readers but only one writer (great for databases).